

Analýza zraniteľností v rámci bug bounty programu

Dominik Sajko

Fakulta informačných technológií

Vysoké učení technické

Brno, Česká republika

xsajko01@fit.vutbr.cz

Abstract—Tento článok prezentuje výsledky, nástroje a postupy penetračného testovania reálne nasadenej webovej aplikácie v rámci bug bounty programu. Cieľom bolo identifikovať a nahlásiť bezpečnostné nedostatky v implementácii webovej aplikácie. Testovanie sa zameriavalo najmä na zraniteľnosti typu: Cross-Site Scripting (XSS) [2], Cross-Site Request Forgery (CSRF) [3] a Improper Access Control [5].

Najvýznamnejším nálezom bola identifikácia zraniteľnosti typu reflected XSS [4], ktorá bola úspešne potvrdená v rámci používateľského účtu. Výsledky poukazujú na nedostatky v implementácii webovej aplikácie, ktorá pracuje s citlivými používateľskými údajmi. Článok zároveň poskytuje návrhy na odstránenie zistených bezpečnostných rizík.

Index Terms—bug bounty, XSS, CSRF, Improper Access Control, webová aplikácia, penetračné testovanie, bezpečnosť

I. ÚVOD

Od začiatku 21. storočia sa svetová ekonomika výrazne presunula do digitálneho priestoru, čo zásadným spôsobom zmenilo spôsob interakcie medzi podnikmi a zákazníkmi. Elektronické obchodovanie prostredníctvom internetových obchodov je dnes jedným z hlavných pilierov digitálnej infraštruktúry, pričom zabezpečuje rýchly a jednoduchý spôsob nakupovania pre milióny používateľov na celom svete. Postupnou digitalizáciou však pribúdajú aj nové kybernetické hrozby. Úspešné útoky môžu viesť k strate citlivých údajov, narušeniu integrity objednávok, reputačným škodám či priamym finančným stratám spoločností. Preto sa kybernetická bezpečnosť stala jednou z najdôležitejších oblastí moderného IT priemyslu.

Internetové obchody predstavujú atraktívny cieľ pre útočníkov najmä z dôvodu priameho kontaktu s bankovými údajmi a osobnými informáciami zákazníkov. Zároveň obsahujú komplexnú logiku používateľských účtov, košíka, objednávok a manažovacích rozhraní, kde akákoľvek chyba v týchto procesoch môže umožniť útočníkovi manipulovať s dátami či prevziať kontrolu nad účtom. Zraniteľnosti webových aplikácií sú práve z tohto dôvodu detailne monitorované a kategorizované v rámci medzinárodných štandardov, ako je napríklad OWASP Top 10 [1], ktorý pravidelne identifikuje najčastejšie a najkritickejšie hrozby v oblasti webovej bezpečnosti.

Medzi najčastejšie chyby patrí nedostatočné ošetrovanie vstupov umožňujúce Cross Site Scripting, chýbajúca kontrola prístupových práv či absencia ochranných mechanizmov proti neoprávneným akciám medzi používateľmi, akými sú útoky Cross site Request Forgery [3]. Tieto zraniteľnosti predstavujú

vážne riziko najmä v prípade systémov pracujúcich s aktívami, ktoré je potrebné udržať v súkromí, ako napríklad osobné a platobné údaje. Útočník tak môže získať prístup k týmto informáciám, zmeniť parametre objednávky alebo priamo obísť autentizačné mechanizmy aplikácie.

Cieľom tohto projektu je prakticky overiť úroveň bezpečnosti v reálne nasadenom internetovom obchode a vyhodnotiť implementáciu obranných mechanizmov proti konkrétnym typom útokov. Testovanie bolo vykonané v rámci programu bug bounty, konkrétne OpenBugBounty [8], ktorý umožňuje etickým hackerom identifikovať zraniteľnosti bez negatívneho dopadu na prevádzku systému. Metodika zahŕňala manuálne aj automatizované testovanie, analýzu jednotlivých častí webovej aplikácie a overenie správania aplikácie pri spracovaní neočakávaných vstupov.

Primárne sa testovanie zameralo na zraniteľnosti typu Cross-Site Scripting (XSS) [2], Cross-Site Request Forgery (CSRF) [3] a Improper Access Control [5]. Zásadným zistením bola identifikácia a potvrdenie zraniteľnosti Reflected XSS [4] vo funkčnosti prihláseného používateľa, ktorá môže viesť prevzatíu kontroly nad užívateľským účtom, krádeži osobných údajov a vykonávaniu akcií bez vedomia užívateľa.

Výsledky testovania poukazujú na potrebu dôkladnejšej implementácie validácie používateľských vstupov v prostredí dynamických webových aplikácií. Súčasťou článku sú tiež odporúčania na minimalizáciu zistených rizík a zhrnutie zistení, ktoré poskytuje informácie o bezpečnostnom stave testovanej webovej aplikácie.

V Kapitole II opisujem metodiku penetračného testovania a použité nástroje. Kapitola III sa venuje samotnému popisu procesu penetračného testovania vrátane analýzy zistených zraniteľností a ich dopadov na bezpečnosť systému. Kapitola IV uzatvára prácu zhrnutím hlavných výsledkov a návrhmi na vylepšenie.

II. METODOLÓGIA

Pred samotným zahájením testovania cieľovej webovej aplikácie bolo nutné najprv vykonať prípravu, aby bola zabezpečená čo najvyššia kvalita výsledkov a minimalizovalo sa riziko porušenia pravidiel bug bounty a etických zásad kybernetickej bezpečnosti. Príprava zahŕňala predovšetkým výber vhodnej webovej aplikácie, ktorá by svojím charakterom a funkcionalitou predstavovala realistický priestor pre overenie bezpečnosti. Súčasťou bolo aj naštudovanie súčasných poznatkov o zraniteľnostiach typických pre webové aplikácie a

zásad zodpovedného odhaľovania zraniteľností, aby všetky kroky prebiehali v striktné etickom a právnom súlade. Posledným krokom bol výber vhodných nástrojov pre systematické preverovanie bezpečnostných slabín v cieľovom systéme.

A. Výber cieľa

Výber cieľa predstavuje kľúčový krok v rámci bezpečnostného výskumu z hľadiska rozsahu testovania a právnych obmedzení. Táto práca vychádza z možností ponúkanými verejne dostupnými bug bounty programmi, ktoré poskytujú bezpečný priestor pre zodpovedné odhaľovanie zraniteľností a zároveň jasne definujú podmienky interakcie s cieľovými systémami.

Ako vhodný cieľ pre túto prácu som zvolil internetový obchod prevádzkovaný nemeckou spoločnosťou Dr. med. Michalziky, ktorý som našiel v bug bounty programe OpenBugBounty [8]. Rozsah tohto bug bounty programu sa vzťahuje na webovú stránku biotikon.de. Výber tohto konkrétneho cieľa bol motivovaný nasledujúcimi faktormi:

- Platforma je súčasťou reálneho obchodného systému, ktorý pracuje s citlivými údajmi zákazníkov.
- Web je zapojený v programe bug bounty.
- Organizácia explicitne povoľuje neinvazívne bezpečnostné testovanie na základe verejne definovaných podmienok.
- Nejedná sa o zastaralý, či neaktívny web.
- Web nepatrí medzi rozsiahle globálne platformy s verejným bug bounty programom, ktoré sú predmetom neustáleho a masívneho testovania.

Testovanie bolo vykonávané v plnom súlade s podmienkami uvedenými prevádzkovateľom:

- Povoľené sú iba neintruzívne útoky.
- Výskum nesmie narušiť dostupnosť služby. Teda nie sú povolené útoky typu DoS a DDoS.
- Zakázané sú akékoľvek formy sociálneho inžinierstva.
- Každý nález musí byť nahlásený súkromne prevádzkovateľovi.

Projekt rešpektuje aj všeobecný právny rámec Českej republiky a európske normy ochrany údajov GDPR [9], keďže webová aplikácia spracúva osobné údaje jej používateľov.

B. Použité nástroje

Pri identifikácii zraniteľností je kľúčové využitie kvalitných analytických nástrojov, ktoré umožňujú podrobné preskúmanie sieťovej komunikácie a odozvy aplikácie na vložené testovacie vstupy. V rámci tohto výskumu bol použitý kombinovaný prístup založený na pasívnej analýze viditeľných komponentov a následnom cielenejšom overovaní bezpečnostných nedostatkov pomocou proxy a automatizovaných testovacích nástrojov.

Hlavným nástrojom použitým pri penetračnom testovaní bol Burp Suite. Jedná sa o profesionálnu platformu určenú na analýzu a testovanie bezpečnosti webových aplikácií. Umožňuje zachytávať, modifikovať a opätovne odosielať

HTTP(S) komunikáciu medzi klientom a serverom, čím poskytuje detailný náhľad na spracovanie vstupov webovou aplikáciou. Jeho architektúra poskytuje širokú škálu funkcionalít, vrátane proxy zachytávania požiadaviek, automatizovaného skenovania vybraných zraniteľností, generovania testovacích payloadov či intrudingu na overovanie odolnosti systémov voči neošetreným vstupom. V rámci testovania slúžil Burp Suite najmä na mapovanie aplikácie, analýzu požiadaviek a odpovedí servera, opetovné odosielanie či upravovanie požiadaviek na praktické overovanie prítomnosti zraniteľností, najmä pri testovaní útokov typu XSS a pri validácii kontrol prístupu [11].

Ďalším nástrojom bol Firefox Developer Tools, ktorý je súčasťou webového prehliadača Firefox. Tento nástroj umožňuje analyzovať zdrojový kód webovej stránky, konkrétne HTML, CSS a JavaScript priamo počas behu aplikácie, sledovať sieťovú komunikáciu a manažovať uložené dáta v prehliadači (Local Storage, Session Storage, Cookies). V rámci vykonávaného testovania bol Firefox Developer Tools využívaný najmä na manuálne overovanie spôsobu, akým aplikácia upravuje dáta zadané užívateľom pred ich zobrazením vo webovom prehliadači, na analýzu DOM štruktúry a monitorovanie odpovedí servera pri interakcii s formulármi. Tento nástroj zároveň uľahčil priamu identifikáciu častí kódu, kde dochádza k dynamickému dopĺňaniu používateľských údajov do stránky, čo bolo kľúčové najmä pri testovaní zraniteľností typu XSS a pri sledovaní manipulácie s parametrami na strane klienta [10].

Ako doplnkový nástroj som využil aplikáciu OWASP ZAP, ktorá slúži na automatizovanú identifikáciu bežných zraniteľností webových aplikácií. Ide o open source bezpečnostný skener vyvíjaný v rámci projektu OWASP, ktorý umožňuje pasívne aj aktívne testovanie bezpečnostných nedostatkov počas komunikácie medzi klientom a serverom. V tejto práci bol OWASP ZAP využitý na automatizované prehľadávanie cieľovej domény a predbežnú identifikáciu potenciálnych slabín, ako sú XSS, problémy s hlavičkami zabezpečenia, nedostatočné obmedzenia prístupových práv alebo chýbajúce validácie vstupov. Tento nástroj som použil najmä na doplnenie manuálneho testovania a validáciu ručne zistených indikácií zraniteľností, pričom výsledky skenov boli vždy následne overené manuálne [12].

Wappalyzer som využil ako nástroj na identifikáciu použitých technológií a ich verzií v cieľovej webovej aplikácii. Tento nástroj umožňuje automaticky detegovať webové frameworky, knižnice, servery, databázy, analytické služby a ďalšie komponenty, ktoré môžu mať vplyv na bezpečnosť aplikácie. Získané informácie zohrávali kľúčovú úlohu pri určovaní vhodných techník testovania a možných vektorov útoku. Napríklad detekcia použitia Shopware, Symfony či ochranných mechanizmov na strane klienta pomohla pri zameraní sa na konkrétne oblasti, kde sa historicky vyskytujú zraniteľnosti. Wappalyzer tak poskytol základný technický prehľad o použitých technológiách, ktorý podporil efektívne plánovanie penetračného testovania [13].

III. PRIEBEH PENETRAČNÉHO TESTOVANIA

Po definovaní metodológie a výbere vhodných nástrojov nasledovala praktická časť penetračného testovania, zameraná na overenie bezpečnostných mechanizmov cieľovej webovej aplikácie. V tejto praktickej časti projektu som využil analytický prístup, kde som testovanie rozdelil do viacerých etáp a jednotlivé typy zraniteľností som testoval samostatne. Testovanie prebiehalo z pohľadu užívateľa stránky, čo malo simulovať reálny scénar, kde útočník vlastní užívateľský účet a snaží sa využiť slabé miesto systému v svoj prospech alebo escalovať svoje privilégia.

- 1) Prvou etapou bol detailný prieskum cieľovej webovej stránky s cieľom pochopiť jej architektúru. S využitím nástroja Wappalyzer som identifikoval technológie využívané na webovej stránke a ich verzie. Následne som manuálne preskúmal dostupné endpointy webovej stránky, pričom všetka HTTP komunikácia bola zaznamenávaná cez proxy server pre neskoršiu hĺbkovú analýzu. Pozornosť som venoval predovšetkým identifikácii všetkých vstupných bodov aplikácie, cez ktoré dochádza k interakcii medzi klientom a serverom. Každý formulár, URL parameter a HTTP hlavička boli považované za potenciálny bod pre injekciu škodlivého kódu. V závere tejto etapy som spustil automatizované testy pomocou programu OWASP ZAP na detekciu nízkourovňových konfiguračných chýb.
- 2) V ďalšej etape som analyzoval zozbierané dát o fungovaní aplikácie a jednotlivých vstupoch používateľa. Zameriam sa na funkcie, ktoré pracujú s osobnými údajmi a systémom nakupovania. Na základe toho som posúdil, ktoré prvky môžu predstavovať vyššie bezpečnostné riziko, stanovil som postup ďalšieho testovania a určil, na ktoré typy útokov a konkrétne časti aplikácie sa zameriam v praktickej fáze.
- 3) V tejto etape boli teoretické predpoklady z analýzy zozbieraných dát overované manuálnymi testami s využitím modulov Burp Suite Proxy a Burp Suite Repeater, ktorý umožnil precíznú manipuláciu s požiadavkami. Testovanie zahŕňalo techniky fuzzingu vstupných polí pre detekciu Cross Site Scripting a manipuláciu identifikátorov objektov v HTTP požiadavkách na odhalenie chýb v riadení prístupu. Osobitný dôraz bol kladený na obchádzanie klientskych validácií a analýzu reakcií servera na neštandardné vstupy. Testovanie jednotlivých zraniteľností je popísané v kapitolách Cross Site Scripting??, Cross Site Request Forgery?? a Improper Access Control??.
- 4) Záverečná etapa zahŕňala zhrnutie zistení. Každá anomália a chyba bola zdokumentovaná formou popisu a prípadne snímok obrazovky, ktoré preukazujú reprodukovateľnosť chyby. Súčasťou výstupu bol aj návrh spôsobu opravy zraniteľných miest.

A. Cross Site Scripting

Prvým typom zraniteľnosti, ktorý som testoval je Cross Site Scripting, Tejto zraniteľnosti som venoval najviac času.

Cieľom bolo analyzovať ako webová aplikácia spracováva používateľské vstupy v rôznych kontextoch ako je vyhľadávanie, úprava profilu alebo práca s nákupným košíkom. Venoval som sa hlavne tomu, či aplikácia správne sanitizuje používateľské vstupy aby sa predišlo vloženiu škodlivého kódu do webovej stránky.

Prvým a najdostupnejším vstupným bodom pre testovanie XSS bolo hlavné vyhľadávacie pole produktov, ktoré je prítomné na každej podstránke. Tento komponent spracováva vstup používateľa a okamžite ho vracia v tele odpovede, čo z neho robí vhodný cieľ pre Reflected XSS.

Po zadaní testovacieho payloadu

```
<script>alert(1)</script>
```

bola analyzovaná HTTP odpoveď servera. Bolo zistené, že aplikácia na strane backendu implementuje kontextové kódovanie výstupu. Špeciálne znaky, ktoré majú v HTML syntaxi riadiaci význam, boli zamenené za bezpečné HTML znaky. Vizuálna reprezentácia na stránke síce zobrazovala vložený skript, avšak v zdrojovom kóde bol reťazec zapísaný nasledovne:

```
"alert(1)" <script>alert(1)</script>
```

Prehliadač tento vstup interpretoval výhradne ako textový obsah, čím bolo zabránené vykonaniu JavaScriptu.

Počas analýzy zdrojového kódu webovej stránky bola identifikovaná potenciálna anomália v bloku JavaScriptu, ktorý inicializuje stav stránky. Server do odpovede vkladá JSON objekt obsahujúci meta-dáta o vyhľadávaní, kde sa vstup používateľa nachádzal v relatívne surovej podobe:

```
{"pageTitle":"0 products found for "  
  <script>alert(1)</script>"",...
```

Tento nález som ďalej testoval, nakoľko vkladanie dát do JavaScriptového kontextu (JavaScript Context) je často náchylné na chyby. Cieľom exploitácie bolo pokúsiť sa "vyskočiť" z atribútu pageTitle ukončením reťazca pomocou úvodzovky a vložením vlastného kódu. Boli vykonané pokusy o injekciu payloadov typu test"; alert(1); //. Avšak, analýza ukázala, že sanitizačná funkcia servera správne ošetrí riadiace znaky. Akýkoľvek pokus o vloženie úvodzovky " bol automaticky sanitizovaný spätným lomítkom na ". Tým pádom nebolo možné narušiť štruktúru JSON objektu a spustiť škodlivý kód.

Ďalším logickým krokom bolo testovanie perzistentných XSS útokov, ktoré sú nebezpečnejšie, nakoľko sa ukládajú do databázy. Cieľom bol formulár pre úpravu osobných údajov v profile používateľa.

Pri pokuse o modifikáciu mena používateľa vstupom:

```
<b>name</b>
```

došlo k zaujímavému správaniu aplikácie. Server nevrátil chybovú hlášku, ale aktívne modifikoval vstup odstránením HTML značiek. Výsledné uložené meno bolo name (bez tučného písma). Ďalšie testy potvrdili, že backendová logika

odstraňuje akýkoľvek text uzavretý v lomených zátvorkách < >.

Zaujímavým zistením bola detekcia anomálie pri spracovávaní užívateľských vstupov. Zatiaľ čo samostatný znak > bol akceptovaný, vstup obsahujúci neuzatvorenú alebo poškodenú značku, napríklad <Text alebo «, spôsobil zlyhanie servera. Aplikácia vrátila generickú chybovú hlášku: "Unfortunately something went wrong". Rovnaký výsledok nastal pri pokuse o uloženie prázdnej značky <>. Táto reakcia indikuje, že sanitizačná knižnica nedokáže korektné spracovať malformované vstupy, čo vedie k vyhodneniu výnimky na strane servera. Hoci toto správanie znemožňuje XSS útok, pretože dáta sa neuložia, poukazuje na problémy s validáciou vstupov.

Najkritickejšia zraniteľnosť bola identifikovaná v module pre úpravu adresy. Tento proces je asynchrónny, teda počas vyplňovania formulára adresy aplikácia najprv overuje existenciu adresy voči externej databáze.

Ak server nedokáže adresu validovať, vráti JSON odpoveď so statusom address_not_found:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "predictions": [],
    "status": ["A3000", "address_not_found"]
  }
}
```

Na základe tejto odpovede skript na strane klienta vygeneruje okno, ktoré žiada používateľa o manuálne potvrdenie správnosti adresy. Toto okno vypisuje údaje zadané používateľom. Práve v tomto bode dochádza k zlyhaniu zabezpečenia. Zatiaľ čo backendová validácia je prísna, frontendová komponenta, ktorá generuje náhľadové okno, vkladá vstupy používateľa priamo do DOM stromu bez sanitizácie.

Pre demonštráciu zraniteľnosti bol do poľa Ulica vložený nasledujúci payload:

Hlavn

Tento payload je navrhnutý tak, aby obišiel jednoduché filtre hľadajúce značku <script>. Využíva HTML element s neplatným zdrojom (src=x). Keď sa prehliadač pokúsi načítať obrázok a zlyhá, automaticky sa spustí event handler onerror, ktorý vykoná funkciu alert(1).

Po vyvolaní validačného dialógu sa škodlivý kód okamžite vykoná.

Jedná sa o bezpečnostnú chybu Reflected XSS, kde chyba vzniká na strane klienta. Útočník by mohol túto chybu zneužiť vytvorením špeciálne upraveného odkazu, ktorý by po kliknutí obete predvyplnil formulár adresy škodlivým kódom a vyvolal validačné okno.

Druhá zraniteľnosť rovnakého typu bola identifikovaná v záverečnej fáze objednávky pri uplatňovaní zľavových kupónov. Testovanie preukázalo, že pole zľavového kupónu nie je dostatočne ošetrované. Po vložení payloadu:

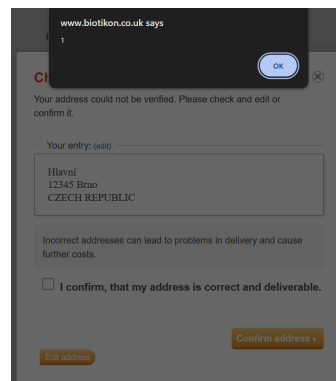


Fig. 1. Ukážka využitia zraniteľnosti XSS pri úprave adresy používateľského účtu.

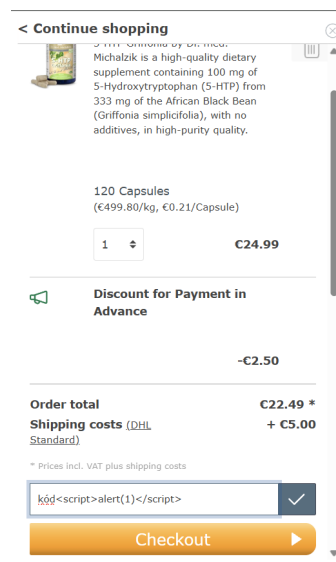


Fig. 2. Zadávanie škodlivého kódu do poľa pre zľavový kupón.

server spracoval požiadavku a vrátil chybovú správu o neplatnom kupóne.

Kritickým zistením je, že chybová správa obsahovala vstup používateľa v neupravenej forme. Server vrátil HTML kód:

```
<div class="alert-content"> Promo code "kod"
could not be found.
</div>
```

V tomto prípade došlo k spusteniu JavaScriptu priamo pri načítaní stránky s chybovou hláškou. Na rozdiel od chyby v úprave adresy, ktorá bola spôsobená logikou na strane klienta, tu ide o zlyhanie validácie na strane servera, a teda o chybu typu Server-side Reflected XSS. Server prijal vstup a bezpečne ho nezakódoval pred vložením do HTML odpovede.

Nájdené zraniteľnosti v module adresára a nákupného košíka sú klasifikované ako Reflected XSS [4]. Hoci sa považujú za menej závažné ako Stored XSS(CITE) pretože

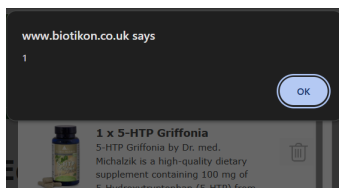


Fig. 3. Ukážka XSS pri zadaní škodlivého kódu do poľa pre zľavový kupón.



Fig. 4. Hlásenie o chybe pri zadaní zlého zľavového kupónu.

vyžadujú priamú interakciu užívateľ'a, v kontexte internetových obchodov predstavujú Vysoké riziko.

Úspešné zneužitie týchto chýb umožňuje útočníkovi:

- **Session Hijacking:** Prístup k premennej `document.cookie` a odcudzenie identifikátora relácie, čo vedie k úplnému prevzatiu účtu obete.
- **Phishing:** Útočník môže pomocou JavaScriptu dynamicky zmeniť obsah stránky a zobrazit' falošný formulár pre zadanie kreditnej karty, pričom používateľ stále vidí v adresnom riadku legitímnu doménu obchodu.
- **Keylogging:** Zaznamenávanie stlačených kláves pri zadávaní citlivých údajov.

B. Cross Site Request Forgery

Pri testovaní odolnosti webovej aplikácie voči útokom typu Cross Site Request Forgery som sa zamerlal na funkcie, ktoré menia stav aplikácie. V kontexte internetového obchodu je kritickým bodom modul pre úpravu adresy používateľ'a, kde by úspešný CSRF útok mohol viesť k presmerovaniu tovaru na adresu útočníka bez vedomia obete.

Report vygenerovaný bezpečnostným skenerom OWASP ZAP poukázal absenciu Anti-CSRF tokenov vo formulároch aplikácie, čo predstavovalo potenciálne riziko zraniteľnosti typu Cross-Site Request Forgery. S cieľom potvrdiť tento nález a vylúčiť možnosť falošného poplachu som vykonal manuálnu analýzu štruktúry HTTP požiadavku, ktorá sa odosiela pri ukladaní upravenej adresy. Bližšie preskúmanie tela požiadavky v nástroji Burp Suite potvrdilo prvé zistenie skenera. Aplikácia nevyužíva explicitný náhodne generovaný `_csrf_token` token.

Absencia Anti-CSRF tokenu v tele požiadavky potvrdila podozrenie na nedostatočnú konfiguráciu čo ma viedlo k domnienke, že server nemá mechanizmus na rozlíšenie legitímnej požiadavky od požiadavky podvrhnutéj z inej domény.

```
POST /account/address/019aa6cdfdc971acb9ef13bb23691ec7
HTTP/2 Host: www.biotikon.co.uk ...
```

```
address%5BfirstName%5D=Alanecsr&...
```

Pre overenie zraniteľnosti bol vykonaný manuálny test v prostredí Burp Suite Repeater. Cieľom bolo simulovať scenár, pri ktorom požiadavka prichádza z externého zdroja a nenesie

so sebou informácie o kontexte používateľ'a, ktoré prehliadač pri Cross Site požiadavkách nemusí odoslať.

Z HTTP požiadavky boli odstránené nasledujúce ochranné hlavičky:

- **Cookie:** Odstránenie identifikátora relácie.
- **Referer:** Odstránenie informácie o pôvodnej stránke.
- **Origin:** Odstránenie informácie o pôvode požiadavky.

Následne bola odoslaná modifikovaná požiadavka s pokusom o zmenu mena na Alanecsr. Server na takto upravenú požiadavku neodpovedal zmenou údajov, teda stavovým kódom 200 OK, ale vrátil stavový kód 302 Found s presmerovaním na prihlasovaciu stránku:

HTTP/2 302 Found Location:

```
/account/login?redirectTo=frontend.account.address.edit.save...
```

Hoci aplikácia neimplementuje Anti-CSRF tokeny, test preukázal, že je voči CSRF útokom v praxi odolná vďaka striktnému manažmentu relácií a spoliehaniu sa na moderné bezpečnostné štandardy prehliadačov.

- 1) **Kontrola relácie:** Server striktné vyžaduje prítomnosť platnej session cookie. Bez nej požiadavku odmieta a odošle klientovi požiadavku s presmerovaním na opätovnú autentifikáciu.
- 2) **SameSite Cookie Atribút:** Moderné prehliadače pri nastavení cookies predvolene aplikujú politiku `SameSite=Lax`. Toto nastavenie zabezpečuje, že prehliadač neodošle session cookie pri POST požiadavke, ktorá prichádza z cudzej domény.

C. Improper Access Control

V nadväznosti na testovanie nákupného procesu bolo vykonané preverenie autorizačných mechanizmov aplikácie. Cieľom bolo overiť odolnosť systému voči útokom typu Improper Access Control. Tento typ zraniteľnosti vzniká, keď aplikácia sprístupňuje priamy odkaz na interný objekt na základe vstupu používateľ'a, pričom zlyhá v overení, či má daný používateľ oprávnenie tento objekt vidieť.

Najprv som sa pri testovaní zamerlal na horizontálnu eskaláciu privilégií, teda snahu získať prístup k dátam inej osoby používateľ'a na rovnakej úrovni oprávnení.

Prvým krokom horizontálnej eskalácie privilégií bola analýza spôsobu, akým aplikácia identifikuje jednotlivé objednávky v URL adresách pri požiadavkách na ich úpravu alebo zobrazenie. Pre tento účel bola vytvorená legitímna testovacia objednávka. Po úspešnom vytvorení objednávky bola v nástroji Burp Suite Proxy zachytená HTTP požiadavka smerujúca na detail tejto objednávky. Analýza URL štruktúry odhalila nasledujúci formát:

```
GET
```

```
/account/order/edit/019aa71815e673cc8810ae442b3a5087
```

```
HTTP/2 Host: www.biotikon.co.uk
```

```
Cookie: session=...
```

Hlavným zistením bolo, že aplikácia pre technickú identifikáciu zdrojov v URL nevyužíva predvídateľné sekvenčné čísla, ktoré sú triviálne na uhádnutie napríklad pomocou

enumerácie v nástroji Burp Suite Repeater. Namiesto toho sú použité unikátne identifikátory s vysokou entropiou.

Zaujímavým bezpečnostným prvkom je oddelenie čísla objednávky od identifikátoru objednávky na stránke. Zatiaľ čo používateľ v rozhraní vidí krátke číslo objednávky, pre akúkoľvek manipuláciu s objednávkou ako je jej editácia sa vyžaduje znalosť dlhého unikátneho identifikátora.

Napriek použitiu silných identifikátorov bol vykonaný pokus o neoprávnený prístup manipuláciou požiadavky v module Burp Suite Repeater. Testovací scenár vychádzal z predpokladu, že útočník by sa mohol pokúsiť pristúpiť k objednávke iného zákazníka znalosťou unikátneho identifikátora danej objednávky.

Boli vykonané nasledujúce testy:

- 1) Manipulácia s unikátnym identifikátorom: Pokus o zmenu posledných znakov v unikátnom identifikátore. Cieľom bolo zistiť, či existuje iná platná objednávka s podobným hashom.
- 2) Force Sequential ID: Pokus o nahradenie unikátneho identifikátora v URL adrese za sekvenčné číslo objednávky, ktoré je verejne známejšie a ľahšie uhádnuť.

GET /account/order/edit/794348 HTTP/2

- 3) Pokus o prístup bez relácie: Odoslanie požiadavky na platný unikátny identifikátor, ale s odstránenou session cookie, pre overenie, či je objekt prístupný verejne.

Všetky pokusy o manipuláciu prístupu boli serverom zamietnuté.

- Pri pokuse o použitie sekvenčného čísla vrátil server chybu, čo potvrdzuje, že endpoint striktné vyžaduje formát unikátneho identifikátora.
- Pri pokuse o zmenu unikátneho identifikátora server vrátil 403 Forbidden.
- Pri pokuse o prístup bez prihlásenia došlo k presmerovaniu na endpoint pre prihlásenie užívateľa.

V druhej časti testovania som sa zameril na overenie odolnosti systému voči vertikálnej eskalácii privilégií. Tento test simuloval snahu bežného autentifikovaného používateľa získať prístup k administratívnejmu rozhraniu webovej aplikácie, ktoré by malo byť vyhradené pre rolu správcu.

V rámci prvej fázy penetračného testovania, ktorou bolo prekúmanie webovej aplikácie som pomocou nástroja Wapalyzer zistil, že aplikácia využíva framework Symfony [6] a obchodnú platformu Shopware [7]. Na základe tohto poznania boli testy zamerané na štandardné administratívne cesty špecifické pre tieto platformy, ako napríklad /admin/*, /api/*, /manager/*.

GET /admin HTTP/2 Host: www.biotikon.co.uk

Cookie: session=...

Pri pokuse o prístup na najbežnejšiu administratívnu cestu, admin server správne detegoval nedostatočné oprávnenia pre reláciu bežného používateľa. Namiesto zobrazenia administratívneho panelu došlo k stavovému kódu 302 Found a následnému presmerovaniu na prihlasovací formulár určený len pre administrátorov.

HTTP/2 302 Found Location:

<https://www.biotikon.co.uk/admin#/login/>

IV. ZÁVER

Cieľom tohto článku bola bezpečnostná analýza reálne nasadenej webovej aplikácie internetového obchodu zapojeného do programu OpenBugBounty [8]. Prostredníctvom kombinácie manuálneho a automatizovaného penetračného testovania bola overená odolnosť aplikácie voči vybraným zraniteľnostiam.

Najvýznamnejším výsledkom práce je identifikácia a potvrdenie dvoch zraniteľností typu Reflected Cross Site Scripting. Prvá zraniteľnosť, spôsobená nedostatočnou sanitizáciou vstupov na strane klienta, bola odhalená v module úpravu adresu používateľa. Druhá zraniteľnosť bola identifikovaná v procese uplatňovania zľavových kupónov, kde server vracal nebezpečný vstup priamo v chybovej hláske. Tieto nálezy potvrdzujú, že aj v moderných internetových obchodoch, môžu vznikať bezpečnostné diery pri implementácii vnútornej logiky alebo dodatočných modulov. Úspešné zneužitie týchto chýb by útočníkovi umožnilo odcudziť session cookies, vykonať phishingové útoky alebo manipulovať s obsahom stránky v kontexte prihláseného používateľa.

Na druhej strane, testovanie v oblastiach riadenia prístupu a ochrany relácie preukázalo odolnosť systému. Aplikácia úspešne odolala pokusom o útoky typu Cross Site Request Forgery a Improper Access Control. Analýza ukázala, že absencia explicitných anti-CSRF tokenov bola nahradená striktnou politikou 'SameSite' atribútov cookies v moderných prehliadačoch. Rovnako použitie náhodne generovaných unikátnych identifikátorov namiesto sekvenčných čísel výrazne sťažilo možnosť enumerácie objektov a neoprávnený prístup k dátam iných zákazníkov.

Na základe vykonaných testov odporúčam prevádzkovateľovi webu implementovať striktnú sanitizáciu všetkých používateľských vstupov ešte pred ich vykreslením v prehliadači, a to ako na strane servera, tak aj v klientskych skriptoch. Vhodným opatrením je nasadenie hlavičky Content Security Policy, ktorá by obmedzila zdroje, z ktorých je možné spúšťať skripty, a tým zmiernila dopad prípadných XSS zraniteľností.

Výsledky tejto práce zdôrazňujú dôležitosť bug bounty programov a kontinuálneho bezpečnostného testovania. Zatiaľ čo automatizovaný skener, ktorý som použil dokázal odhaliť absenciu bezpečnostných hlavičiek, kritické logické chyby v spracovaní vstupov boli odhalené až manuálnou analýzou. Všetky identifikované zraniteľnosti boli v súlade s etickými pravidlami reportované prostredníctvom platformy OpenBugBounty [8].

REFERENCES

- [1] OWASP, "OWASP Top 10 Web Application Security Risks," OWASP. [Online]. Available: <https://owasp.org/Top10/> [Accessed: 23-Nov-2025].
- [2] PortSwigger, "Cross-Site Scripting (XSS) Web Security Academy," PortSwigger. [Online]. Available: <https://portswigger.net/web-security/cross-site-scripting> [Accessed: 23-Nov-2025].
- [3] PortSwigger, "Cross-Site Request Forgery (CSRF) Web Security Academy," PortSwigger. [Online]. Available: <https://portswigger.net/web-security/csrf> [Accessed: 23-Nov-2025].

- [4] PortSwigger, "Reflected cross-site scripting (XSS) Web Security Academy," PortSwigger. [Online]. Available: <https://portswigger.net/web-security/cross-site-scripting/reflected> [Accessed: 23-Nov-2025].
- [5] MITRE, "CWE-284: Improper Access Control," Common Weakness Enumeration (CWE). [Online]. Available: <https://cwe.mitre.org/data/definitions/284.html> [Accessed: 23-Nov-2025].
- [6] Symfony, "Symfony, A PHP Framework for Web Applications," Symfony. [Online]. Available: <https://symfony.com/> [Accessed: 23-Nov-2025].
- [7] Shopware, "Shopware E-Commerce Software," Shopware. [Online]. Available: <https://www.shopware.com/en/> [Accessed: 23-Nov-2025].
- [8] OpenBugBounty, "OpenBugBounty, Responsible Disclosure Platform," OpenBugBounty. [Online]. Available: <https://www.openbugbounty.org/> [Accessed: 23-Nov-2025].
- [9] GDPR-Info.eu, "General Data Protection Regulation (GDPR) Legal Text," GDPR-Info.eu. [Online]. Available: <https://gdpr-info.eu/> [Accessed: 23-Nov-2025].
- [10] Mozilla, "Firefox DevTools User Docs," Firefox Source Docs. [Online]. Available: <https://firefox-source-docs.mozilla.org/devtools-user/> [Accessed: 23-Nov-2025].
- [11] PortSwigger, "Burp Suite Vulnerability Scanner," PortSwigger. [Online]. Available: <https://portswigger.net/burp> [Accessed: 23-Nov-2025].
- [12] OWASP, "OWASP ZAP, Zed Attack Proxy," OWASP. [Online]. Available: <https://www.zaproxy.org/> [Accessed: 23-Nov-2025].
- [13] Wappalyzer, "Technology Profiler," Wappalyzer. [Online]. Available: <https://www.wappalyzer.com/> [Accessed: 23-Nov-2025].